SMS Spam Detection Using an LSTM Neural Network

SE4050 - Deep Learning | Individual implementation report | 10 October 2026

Student	Details
Name	Lakshan RHR
Registration number	IT22917416
Email	IT22917416@my.sliit.lk

Abstract

This study implements a binary SMS classifier using a learned embedding and a single long short-term memory (LSTM) network. The recorded local run retained 5,159 usable messages after cleaning and divided them into stratified training, validation and test sets. On 774 held-out test messages, the model achieved 96.90% accuracy, 81.58% spam precision, 96.88% spam recall and 88.57% spam F1. It correctly detected 93 spam messages, missed three, and incorrectly flagged 21 legitimate messages. Validation-based early stopping selected epoch 6. These results indicate promising performance on this split, but one architecture, one seed and an older English benchmark limit broader conclusions.

Scope and evidence statement

This is a single-model implementation. The supplied assignment brief requires at least four distinct supervised deep-learning models; lecturer approval for a reduced scope has not been established. Consequently, this report does not claim to fulfil the model-comparison requirement. No historical weekly contributions, video publication, Turnitin check or course submission are asserted.

Evidence comprises the supplied results.json, config.json, confusion_matrix.png, learning_curve.png and console training log. Source code was reviewed. The recorded local run was not independently reproduced, and the saved checkpoint and prediction transcript were not supplied for this report. No results are invented.

1. Introduction and problem definition

SMS spam detection assigns an incoming message to either spam or ham (legitimate communication). The practical objective is to identify unwanted messages while avoiding interference with legitimate communication. False negatives allow spam through; false positives can hide important messages. The project therefore evaluates both detection coverage and the reliability of spam flags, rather than relying only on overall accuracy.

The research question is: how does one compact LSTM classifier perform on a deduplicated, stratified split of the UCI SMS Spam Collection? The outcome is a reproducible training script, recorded evaluation and command-line prediction interface. This is an experimental classifier, not a deployed message-blocking service.

2. Background and related work

The UCI SMS Spam Collection provides labeled messages for mobile spam research [1]. It offers an accessible benchmark, although its age limits claims about present-day messages. Hochreiter and Schmidhuber introduced LSTM to address difficulties in learning long-term dependencies [2]. An LSTM processes an ordered sequence with gated memory, motivating its use for text classification. That motivation does not establish an empirical advantage on this dataset.

This implementation learns embeddings from its training split rather than importing pretrained representations. PyTorch supplies the recurrent layer and optimization components. Its BCEWithLogitsLoss combines sigmoid and binary cross-entropy in one numerically stable operation and supports positive-class weighting [3]. A TF-IDF linear classifier would be a useful simpler baseline, while CNN and other recurrent architectures would enable direct comparisons. None was trained in this study, so no comparative performance is claimed.

3. Dataset description and exploratory analysis

Source: UCI SMS Spam Collection, credited to Tiago Almeida and Jose Maria Gomez Hidalgo; DOI 10.24432/C5CC84 [1]. The repository lists a CC BY 4.0 license. Each usable record has a categorical label (ham/spam) and a free-text message. The target encoding is ham=0 and spam=1. The parser in the recorded run read 5,574 rows; the local measurement is used here rather than assuming a repository headline count.

Partition	Ham	Spam	Total
Training	3,162	449	3,611
Validation	677	97	774
Test	678	96	774
Clean total	4,517	642	5,159

Spam forms 12.44% of the clean data. Cleaning removed 415 rows in aggregate (7.45% of parsed rows). Separate counts of duplicate, empty and conflicting-label removals were not logged; this total must not be described as exclusively duplicate removal. Detailed length distributions and raw-data examples were not supplied, so exploratory analysis is limited to observed class balance and cleaning counts.

4. Data preprocessing and feature engineering

The pipeline lowercases messages and extracts alphanumeric runs and punctuation tokens. Each message receives a normalized token-sequence key. Keys with conflicting labels and empty keys are excluded; repeated keys are deduplicated before splitting. This avoids identical normalized model inputs in different partitions. It does not guarantee that near-duplicate campaign templates or related messages remain in one partition.

A stratified split with random seed 42 allocates approximately 70% to training and 15% each to validation and test. Vocabulary construction uses training messages only, a minimum token frequency of two and an upper limit of 8,000 entries. The recorded vocabulary contains 3,162 entries including padding and unknown-token symbols. Messages are limited to 100 tokens, and shorter sequences are padded. Packed sequences allow the LSTM to process their actual lengths. The embedding is learned during training; no external pretrained model is used.

5. Experimental design

Setting	Recorded configuration
Embedding / hidden dimensions	64 / 64
Dropout / batch size	0.3 / 64
Optimizer / learning rate	Adam / 0.001
Loss	Weighted binary cross-entropy with logits
Positive-class weight	3,162 / 449 = 7.0423 (training labels only)
Gradient clipping	Maximum gradient norm 1.0
Training limit / early stopping	15 epochs maximum; patience 3
Selection criterion	Lowest validation loss; epoch 6 selected
Decision threshold	Fixed sigmoid score threshold 0.5
Execution	CPU; two threads; seed 42
Recorded environment	Windows, Python 3.12.4; PyTorch 2.14.1+cpu

The test set is evaluated after restoring the selected validation checkpoint. No test-based threshold or hyperparameter tuning is implemented. The run stopped at epoch 9. Recorded training-loop duration was 21.73 seconds; this excludes downloading and final evaluation and is not a general hardware benchmark. CPU model, memory and exact versions of all dependencies were not recorded in the supplied evidence.

Precision, recall and F1 are reported per class, together with accuracy, macro averages, ROC-AUC and a confusion matrix. Positive-class weighting addresses imbalance during optimization. The score threshold remains fixed; scores are not established as calibrated probabilities.

6. Model architecture

Layer	Output / purpose	Parameters
Embedding	Token IDs to 64-dimensional vectors	202,368
One LSTM layer	64-dimensional final hidden state	33,280
Dropout	Randomly zero hidden features in training	0
Linear output	One spam logit	65
Total	Trainable model parameters	235,713

The embedding contains 3,162 x 64 parameters. The LSTM maintains gated state while reading the token sequence. Its input, forget and output gates control information flow, while a candidate state updates memory. Dropout regularizes the final hidden representation. The linear layer generates a logit. Sigmoid is used to convert the logit to a score for prediction; the training loss already incorporates sigmoid [3].

Small embedding and hidden dimensions keep CPU training practical. A single recurrent layer limits implementation complexity. These are reasonable design choices for a compact prototype, but their optimality was not tested. The 100-token cutoff may discard useful information from longer messages.

7. Results and model comparison

The following are results for the same selected model on three partitions. They are not comparisons between different architectures. Values are calculated from the uploaded evaluation file.

Partition	Accuracy	Spam precision	Spam recall	Spam F1	ROC-AUC
Training	99.14%	95.04%	98.22%	96.60%	0.9968
Validation	95.99%	78.95%	92.78%	85.31%	0.9814
Test	96.90%	81.58%	96.88%	88.57%	0.9872

Test macro F1 is 93.39%, while weighted F1 is 97.01%. The weighted average gives more influence to ham because ham is the majority class. Spam F1 therefore adds important information that is less apparent from weighted F1 alone.

For context, a rule that predicts every test message as ham would achieve 678/774 = 87.60% accuracy and zero spam recall. The LSTM improves accuracy over that analytical rule by 9.30 percentage points. This arithmetic reference is not a separately trained baseline and does not satisfy the required multi-model comparison.

7.1 Test confusion matrix

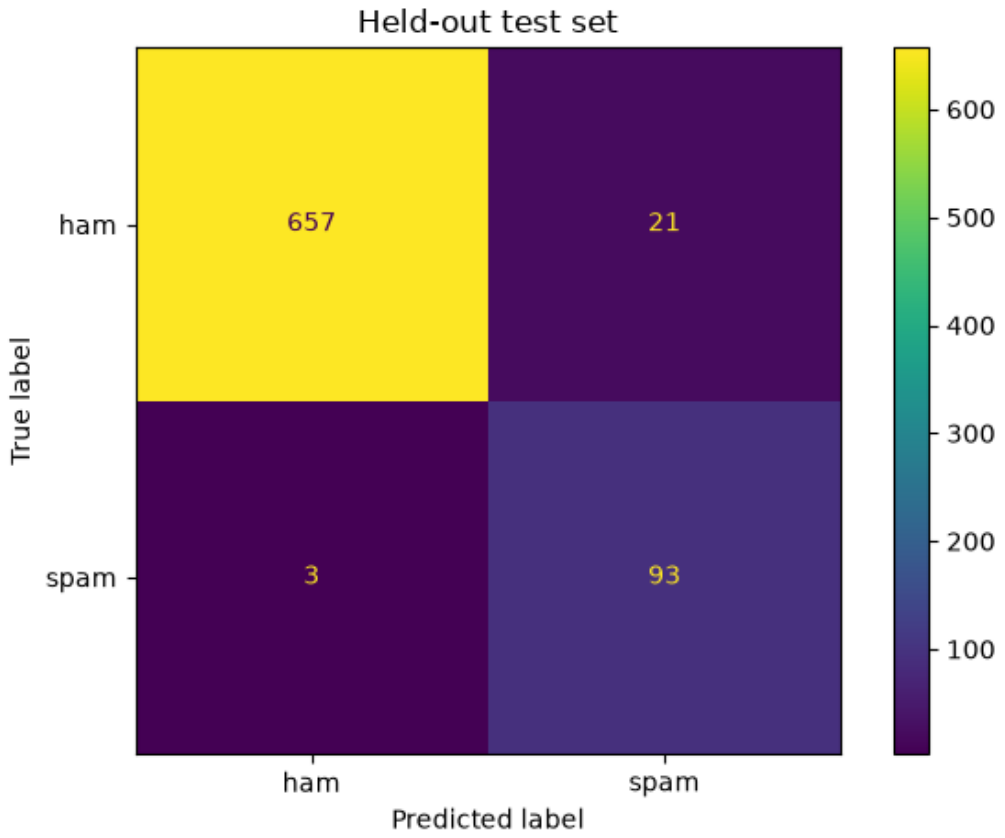


Figure 1. Recorded held-out test confusion matrix. Rows show actual labels; columns show predicted labels.

With spam as the positive class, TP=93, TN=657, FP=21 and FN=3. Thus 750 of 774 predictions are correct. Spam precision is 93/(93+21)=81.58%; spam recall is 93/(93+3)=96.88%. Of the 114 messages flagged as spam, 21 (18.42%) are legitimate. The false-positive rate among legitimate messages is 21/678=3.10%; these two percentages have different denominators.

8. Critical analysis and discussion

Error tradeoff. High recall suggests the model captures most spam in this sample. However, the 21 false alarms matter if the classifier automatically blocks messages. A review or quarantine interface would be more cautious than irreversible blocking. Class weighting may help explain the high-recall operating point, but an unweighted ablation was not performed, so causality cannot be established.

Generalization. Training spam F1 is 96.60%, versus 88.57% on test: a gap of 8.03 percentage points. Training accuracy similarly exceeds test accuracy by 2.24 points. These gaps are consistent with imperfect generalization. The test set contains only 96 spam messages, so each missed spam changes recall by approximately 1.04 percentage points. Multiple seeds or uncertainty estimates are needed before treating this result as stable.

8.1 Training behavior and limitations

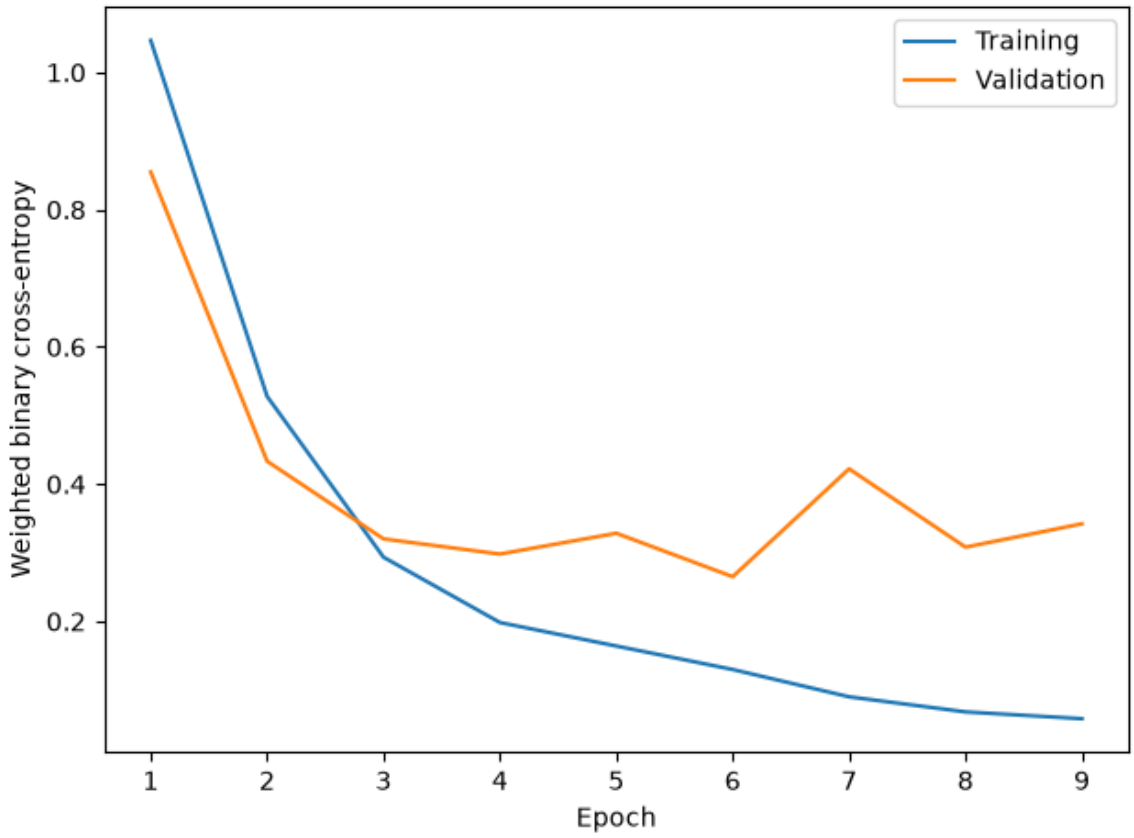


Figure 2. Recorded weighted training and validation loss over nine epochs.

Validation loss reaches its minimum at epoch 6 (0.2652). It rises to 0.4223 at epoch 7 and remains above the minimum at epochs 8 and 9. Training loss continues to fall. This pattern supports early stopping and suggests overfitting risk. Training loss is collected while dropout is active and weights change within an epoch; validation loss is calculated in evaluation mode. Their absolute values are therefore not perfectly comparable.

Efficiency and complexity. A 235,713-parameter model trained in 21.73 seconds in the recorded run. This supports feasibility on the tested laptop, but there are no comparable model timings, memory measurements or latency measurements. It does not establish that LSTM is the most efficient solution.

Limits. The experiment uses one seed and one split, no trained baseline and no architecture comparison. Older English SMS data may not represent modern scams or Sinhala/Singlish messages. Exact normalized deduplication leaves possible template leakage. No message-level error examples or calibration study were provided. Interactive inference was implemented but its successful execution was not evidenced. No evidence supports production readiness.

8.2 Improvements and future work

First, implement distinct supervised architectures under shared splits and comparable tuning budgets, and include a TF-IDF baseline. This would address both the scientific comparison gap and the assignment requirement. Second, group related message templates before splitting and evaluate multiple random seeds to assess sensitivity. Third, use validation data to investigate decision thresholds and precision-recall curves; reserve a new untouched evaluation set if test observations guide further redesign.

Fourth, inspect the actual false-positive and false-negative messages to identify recurring failure modes. Fifth, test on a recent external dataset with the intended language distribution. Report PR-AUC, calibration and inference latency where relevant. These are proposed experiments, not completed work.

9. Conclusion

The recorded LSTM experiment achieved 96.90% accuracy and 88.57% spam F1 on 774 held-out messages. Its strongest measured property is spam recall (96.88%), with a practical cost of 21 legitimate messages flagged as spam. Validation-based checkpoint selection limited continued training after signs of overfitting. The experiment is a functioning single-model starting point; comparative evaluation, broader robustness testing and remaining submission materials are still needed.

10. References

[1] T. Almeida and J. M. Gomez Hidalgo, SMS Spam Collection, UCI Machine Learning Repository, 2011. DOI: 10.24432/C5CC84. Dataset and license: https://archive.ics.uci.edu/dataset/228/sms+spam+collection (accessed 10 October 2026).

[2] S. Hochreiter and J. Schmidhuber, Long Short-Term Memory, Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997. DOI: 10.1162/neco.1997.9.8.1735.

[3] PyTorch, BCEWithLogitsLoss documentation. https://docs.pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html (accessed 10 October 2026).

[4] Local experiment evidence supplied by the student: results.json, config.json, confusion_matrix.png, learning_curve.png and console training log. These provide the empirical results in this report.

Acknowledgement and contribution record

ChatGPT assisted with source-code generation, documentation, interpretation and report preparation. The student executed training locally and supplied its outputs. Other contributions should be described only after they are actually performed. The report should be reviewed by the named student before submission. No past commit history or additional group participation is asserted.